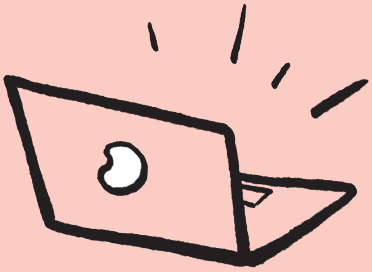

해설 노트

2장

1분 퀴즈

1. ③ System.out.println()은 소괄호 안 값을 화면에 출력하는 명령어입니다. 프로그램의 실행 결과를 보여주는 가장 기본적인 방법입니다.
2. ③ String은 큰따옴표("")로 감싼 문자열 값을 저장하는 자료형입니다. 이름, 문장, 메시지 등 텍스트 데이터를 다룰 때 사용합니다.
3. ① final 키워드를 사용하면 해당 변수는 상수가 되어 이후에 값을 변경할 수 없습니다.
4. ② 주석은 컴퓨터가 실행하지 않고 무시하는 코드 영역입니다. 코드의 의도를 설명하거나 임시로 실행을 막을 때 사용합니다.
5. ④ 실수 값을 정수형으로 명시적 형 변환하면 소수점 이하 부분은 반올림되지 않고 그대로 버려집니다. 예를 들어 98.8은 98이 됩니다.

실습 문제

chap_02/_Quiz_02.java

```
public class _Quiz_02 {
    public static void main(String[] args) {
        // 활동 내용
        String activity1 = "기상, 아침 식사"; ------------------------- ❶
        String activity2 = "자바 학습";
        String activity3 = "휴식";
        // 시작 시간
        int startTime1 = 7; ------------------------------------------ ❷
        int startTime2 = 8;
        int startTime3 = 11;
        // 결과 출력
        System.out.println("--- 나의 하루 계획 ---");
        System.out.println(startTime1 + "시: " + activity1); ------- ❸
        System.out.println(startTime2 + "시: " + activity2);
        System.out.println(startTime3 + "시: " + activity3);
    }
}
```

각 계획은 시작 시간과 활동 내용으로 이루어져 있으므로 이 두 정보를 각각 변수로 저장합니다.

❶ "기상, 아침 식사"처럼 여러 글자로 이루어진 데이터는 문자열에 해당합니다. 따라서 문자열 값을 저장하는 String 자료형을 사용하고, 값은 반드시 큰따옴표(")로 감싸줍니다. 여러 활동을 구분하기 위해 변수 이름 끝에 activity1, activity2, activity3처럼 숫자를 붙여 구분합니다.

❷ 7시와 같은 시간 정보는 소수점이 없는 숫자 값이므로 정수 값을 저장하는 int 자료형을 사용합니다. startTime1이라는 이름의 변수를 만들고, 그 안에 정수 값 7을 저장합니다. 활동 내용과 마찬가지로 여러 시작 시간을 관리하기 위해 변수 이름 끝에 숫자를 붙여 구분합니다.

❸ 변수에 저장된 값들을 조합해 원하는 형식으로 출력하는 역할을 합니다. + 연산자는 문자열과 함께 사용하면 문자열을 연결하는 역할을 합니다. 따라서 이 문장은 다음과 같은 순서로 처리됩니다.

- startTime1에 저장된 정수 값 7
- 문자열 "시: "
- activity1에 저장된 문자열 "기상, 아침 식사"

세 값이 차례대로 연결되어 최종적으로 7시: 기상, 아침 식사 문자열이 만들어지고, 화면에 출력됩니다.

셀프체크

chap_02/_SelfCheck_02.java

```
public class _SelfCheck_02 {
    public static void main(String[] args) {
        String name = "나도코딩";          // 1. 이름
        int steps = 10005;                // 2. 총 걸음 수
        double distance = 7.5;            // 3. 총 이동 거리
        boolean isGoalReached = true;     // 4. 목표 달성 여부
        // 5. 결과 출력
        System.out.println("사용자 이름: " + name);
        System.out.println("오늘 걸음 수: " + steps + "보");
        System.out.println("이동 거리: " + distance + "km");
        System.out.println("목표 달성 여부: " + isGoalReached);
    }
}
```

3장

1분 퀴즈

1. ② 나머지 연산자(%)는 두 수를 나눈 뒤 남는 나머지 값을 계산할 때 사용합니다. 정수 나눗셈에서는 소수점 이하를 버리고 몫만 계산하지만, % 연산자는 그 나눗셈 이후에 남은 값을 구합니다. 따라서 10 % 3 = 1입니다.
2. ② ++value는 변수의 값을 먼저 1 증가시킨 뒤 그 값을 사용하고, value++는 현재 값을 먼저 사용한 뒤 변수의 값을 1 증가시킵니다. 두 연산자 모두 변수의 값은 1 증가하지만, 값을 사용하는 시점이 다르다는 점이 주요 차이점입니다.
3. ① +=는 변수에 값을 더한 뒤 그 결과를 다시 같은 변수에 대입하는 복합 대입 연산자입니다. num += 5;는 num = num + 5;와 완전히 동일하게 작동합니다.
4. ③ = 연산자는 변수에 값을 대입(저장)할 때 사용합니다. 반면 == 연산자는 두 값이 서로 같은지를 비교해 그 결과를 true 또는 false로 반환합니다.
5. ④ OR 연산자(||)는 연결된 조건 중 하나라도 참이면 전체 결과가 참이 됩니다.

실습 문제

chap_03/_Quiz_03.java

```
public class _Quiz_03 {
    public static void main(String[] args) {
        // 세 사람의 몸무게를 int 변수에 저장
        int weight1 = 68; // 단위 kg
        int weight2 = 75;
        int weight3 = 72;
        int tempMaxWeight = (weight1 > weight2) ? weight1 : weight2; ---------- ❶
        int loserWeight = (tempMaxWeight > weight3) ? tempMaxWeight : weight3; --- ❷
        System.out.println("꼴찌의 몸무게는 " + loserWeight + "kg입니다."); ------ ❸
    }
}
```

이 문제는 한 번에 세 값을 비교하려고 하면 복잡할 수 있습니다. 하지만 문제를 두 단계로 나누면 삼항 연산자를 이용해 훨씬 간단하게 해결할 수 있습니다.

❶ **첫 번째와 두 번째 사람의 몸무게 비교**

- weight1(68)이 weight2(75)보다 큰지 비교합니다. 조건의 결과는 false이므로 : 뒤에 있는 weight2를 선택합니다. 따라서 tempMaxWeight에는 75를 저장합니다.
- 이 시점에서 tempMaxWeight에는 첫 번째와 두 번째 사람 중 더 무거운 몸무게가 담겨 있습니다.

❷ **중간 결과와 세 번째 사람의 몸무게 비교**

- tempMaxWeight(75)가 weight3(72)보다 큰지 비교합니다. 결과는 true이므로 ? 뒤에 있는 tempMaxWeight를 선택합니다.
- 그 결과로, loserWeight 변수에 75를 저장합니다.

❸ 두 단계를 거친 결과, loserWeight에는 75가 저장되고 이 값을 화면에 출력합니다.

셀프체크

chap_03/_SelfCheck_03.java

```
public class _SelfCheck_03 {
    public static void main(String[] args) {
        // 1. 초기 데이터 설정
        int price = 30000;
        int coupon = 5000;
        // 2. 최종 금액 계산
        int totalPrice = price - coupon;
        // 3. 무료 배송 조건(2만 원 이상 결제 시)
        boolean applyFreeShipping = (totalPrice >= 20000);
        // 4. 결과 출력
        System.out.println("최종 결제 금액: " + totalPrice + "원");
        System.out.println("무료 배송 적용 여부: " + applyFreeShipping);
    }
}
```

4장

1분 퀴즈

1. ③ length( )는 문자열이 가지고 있는 문자의 개수, 즉 문자열의 길이를 정수(int) 형태로 알려줍니다. 전체 길이에는 공백을 포함하며, 사용자 입력 길이를 제한하거나 문자열을 처리할 때 자주 활용합니다.

2. ① contains()는 문자열 안에 지정한 문자열이 포함되어 있으면 true, 포함되어 있지 않으면 false를 결과로 알려줍니다.

3. ② substring()은 문자열의 시작 인덱스(와 선택적으로 끝 인덱스)를 지정해 원본 문자열에서 원하는 부분만 추출할 때 사용합니다.

4. ② == 연산자는 두 변수가 같은 메모리 공간을 가리키는지 비교합니다. 문자열의 내용이 같은지 비교할 때는 equals() 메서드를 사용해야 합니다.

5. ③ \n은 줄 바꿈을 의미하는 이스케이프 문자로, 문자열을 출력할 때 커서를 다음 줄의 맨 앞으로 이동시키는 역할을 합니다.

실습 문제

chap_04/_Quiz_04.java

```
public class _Quiz_04 {
    public static void main(String[] args) {
        String encodedText = "별들이 제자리를 찾을 무렵, --START--숨겨진 [KEYWORD]의
위치를 기억하라--END--"; ---------------------------------------------------------- ❶
        int msgStartIndex = encodedText.indexOf("--START--") + "--START--".
length(); ------------------------------------------------------------------------- ❷
        int msgEndIndex = encodedText.indexOf("--END--"); ---------------------- ❸
        String originalMessage = encodedText.substring(msgStartIndex,
msgEndIndex); ---------------------------------------------------------------------- ❹
        String decodedMessage = originalMessage.replace("[KEYWORD]", "지혜의 샘");
------------------------------------------------------------------------------------ ❺
        System.out.println("[암호 해독 결과]"); ---------------------------------- ❻
        System.out.println("원본 암호: " + originalMessage);
        System.out.println("해독된 문장: " + decodedMessage);
    }
}
```

❶ 주어진 암호문을 encodedText 라는 이름의 String 변수에 저장합니다.

❷ indexOf()로 "--START--" 문자열이 처음 등장하는 위치를 알아냅니다. 추출하고 싶은 내용은 "--START--" 바로 뒤부터이므로 "--START--" 문자열의 길이만큼을 더합니다. 이때 문자열 변수뿐 아니라 문자열에도 직접 .length() 형태로 바로 사용할 수 있습니다.

❸ indexOf()로 "--END--" 문자열이 시작되는 위치를 구합니다.

❹ 앞에서 계산한 시작과 끝 위치를 substring()에 넣으면, 두 표식 사이에 숨겨져 있던 비밀 메시지가 추출됩니다. substring(시작, 끝)은 끝 위치 바로 앞까지만 잘라내므로 "--END--"의 시작 위치를 그대로 끝 인덱스로 사용하면 원하는 범위를 정확히 추출할 수 있습니다.

❺ replace()로 비밀 메시지 안의 "[KEYWORD]"를 "지혜의 샘"으로 교체합니다. 문자열은 불변이므로 결과를 새 변수에 저장합니다.

❻ System.out.println()과 + 연산자를 사용해 문제에서 요구한 형식대로 결과를 출력합니다.

셀프체크

chap_04/_SelfCheck_04.java

```
public class _SelfCheck_04 {
    public static void main(String[] args) {
        // 1. 이메일 주소 변수 선언
        String email = "nadocoding@gmail.com";
        // 2. '@' 기호의 위치 찾기
        int index = email.indexOf("@");
        // 3. substring을 이용한 문자열 분리
        String id = email.substring(0, index); // 0부터 index 전까지 추출(아이디)
        String domain = email.substring(index + 1); // index + 1부터 끝까지 추출(도메인)
        // 4. 결과 출력
        System.out.println("이메일 아이디: " + id);
        System.out.println("이메일 도메인: " + domain);
    }
}
```

5장

1분 퀴즈

1. ③ if 문은 조건이 참일 때만 중괄호 안 코드를 실행해 프로그램의 흐름을 제어합니다. 조건식이 거짓이면 해당 코드는 실행하지 않고 건너뜁니다.

2. ④ if 문의 조건이 참일 때 실행할 문장이 여러 개라면 이를 중괄호로 묶어 하나의 코드 블록으로 만들어야 합니다.

3. ③ else 문은 if 문의 조건이 거짓(false)일 경우 실행할 코드를 정의할 때 사용합니다.

4. ② else if는 if 문의 조건이 거짓일 경우 이어지는 여러 조건을 순차적으로 검사해 참인 조건에 해당하는 코드만 실행할 때 사용합니다.

5. ④ switch-case 문은 변수의 값이 명확한 여러 값 중 하나와 일치하는지 검사할 때 사용합니다.

6. ④ switch-case 문에서 break를 생략하면 조건에 일치하는 case부터 그 아래에 있는 모든 case의 코드가 순차적으로 실행됩니다. 이러한 동작을 **폴스루**(fall-through) 현상이라고 합니다.

실습 문제

chap_05/_Quiz_05.java

```
public class _Quiz_05 {
    public static void main(String[] args) {
        // 주어진 조건 변수들(값을 바꿔 테스트해 보세요.)
        // 회덮밥 정식 25,000원/불고기 피자(L) 32,000원/모듬회(대) 99,000원
        String menu = "회덮밥 정식"; // 주문 메뉴 ---------------------- ❶
        int foodPrice = 25000;       // 음식 가격 -------------------- ❷
        boolean isRaining = false;    // 비가 오는지 여부 ------------ ❸
        boolean hasVIPCoupon = false; // VIP 쿠폰 소지 여부 ---------- ❹
        int deliveryFee = 3000; ------------------------------------- ❺
        if (isRaining) { ------------------------------------------- ❻
            deliveryFee += 1000;
        }
        if (foodPrice > 70000) { ----------------------------------- ❼
            deliveryFee = 0;
        }
        if (hasVIPCoupon) { ---------------------------------------- ❽
            deliveryFee = 0;
        }
        int totalPrice = foodPrice + deliveryFee; // 최종 결제 금액 계산
        System.out.println("주문 메뉴: " + menu);
        System.out.println("주문 금액: " + foodPrice + "원");
        System.out.println("배달비: " + deliveryFee + "원");
        System.out.println("최종 결제 금액: " + totalPrice + "원");
    }
}
```

주문 메뉴(❶), 음식 가격(❷), 우천 여부(❸), VIP 쿠폰 소지 여부(❹)를 각각 변수로 선언합니다.

❺ 어떤 조건에도 해당하지 않을 때 적용할 기본 배달비(3,000원)를 deliveryFee 변수에 저장합니다.

❻ isRaining 변수의 값이 true이면 비가 오는 날이므로 우천 할증으로 기본 배달비에 1,000원을 추가합니다.

❼ 주문 금액이 70,000원을 초과하면 이전에 계산한 배달비와 관계없이 배달비를 0원으로 설정합니다.

❽ VIP 쿠폰이 있다면 주문 금액이나 날씨와 관계없이 배달비를 0원으로 설정합니다. 이 조건을 가장 마지막에 검사하므로 비가 오거나 주문 금액이 70,000원 미만이더라도 VIP 쿠폰이 있으면 최종 배달비는 항상 0원이 됩니다.

이처럼 if 문의 순서는 프로그램의 최종 결과에 직접적인 영향을 미칩니다. 우선순위가 높은 조건(VIP 쿠폰)을 마지막에 배치해 앞선 계산 결과를 덮어쓰는 구조로 설계한 점이 이 코드의 핵심입니다.

셀프체크

chap_05/_SelfCheck_05.java

```
public class _SelfCheck_05 {
    public static void main(String[] args) {
        int age = 21; // 1. 나이
        boolean isLocalResident = true; // 2. 지역 주민 여부
        // 3. 나이에 따른 기본 요금 산정
        int fee = 0;
        if (age < 13) {
            fee = 10000;
        } else if (age < 19) {
            fee = 20000;
        } else {
            fee = 30000;
        }
        // 4. 지역 주민 할인 적용
        if (isLocalResident) {
            System.out.println("지역 주민 할인이 적용되었습니다.");
            fee = (int) (fee * 0.5);
        }
        // 5. 결과 출력
        System.out.println("최종 결제 금액은 " + fee + "원입니다.");
    }
}
```

6장

1분 퀴즈

1. ① for 문은 for (초기화; 조건식; 증감식) 구조로 구성되며, 세 요소가 반복 횟수와 실행 흐름을 제어합니다.
2. ④ while 문은 조건이 참인 동안 반복을 계속하므로 반복 횟수를 사전에 알 수 없는 경우에 적합합니다.
3. ① while 문은 조건을 먼저 검사하지만, do-while 문은 코드를 먼저 실행한 뒤 조건을 검사합니다. 따라서 조건이 처음부터 거짓이어도 do-while 문은 최소 한 번 실행됩니다.
4. ③ break는 반복문 자체를 즉시 종료하지만, continue는 현재 반복에서 남은 코드를 실행하지 않고 다음 반복을 위한 단계로 이동합니다.

실습 문제

chap_06/_Quiz_06.java

```
public class _Quiz_06 {
    public static void main(String[] args) {
        // 주어진 조건 변수들(전체 대여 기간을 바꿔서 테스트해 보세요)
        int totalDays = 10;     // 전체 대여 기간 -------------- ❶
        int rentalPeriod = 7;   // 정상 대여 기간 -------------- ❷
        int fine = 0;           // 누적 연체료 ----------------- ❸
        System.out.println("===================");
        System.out.println("스마트 도서관 알리미");
        System.out.println("===================");
        for (int day = 1; day <= totalDays; day++) { -------- ❹
            if (day <= rentalPeriod) { ----------------------- ❺
                int remainingDays = rentalPeriod - day;
                System.out.println(day + "일 차: 정상 대여 중(반납까지 "
                        + remainingDays + "일 남음)");
            } else { ----------------------------------------- ❻
                fine += 500; // 하루에 500원씩 추가
                System.out.println(day + "일 차: [연체] 누적 연체료: " + fine + "원");
            }
        }
        System.out.println("===================");
        System.out.println("최종 안내: 미반납 시 총 연체료는 " + fine + "원입니다.");
    }
```

```
}
```

전체 대여 기간(❶), 정상 대여 기간(❷), 누적 연체료(❸)를 각각 변수로 선언합니다.

❹ for 문으로 대여 1일차(day)부터 전체 대여 기간(totalDays)까지 하루씩 증가시키며 대여 상태를 출력합니다.

❺ 현재 날짜(day)가 정상 대여 기간인 7일(rentalPeriod)보다 작거나 같다면 정상 대여 중임을 알려주고, 남은 반납 기한을 계산해 보여줍니다. 남은 날짜는 정상 대여 기간에서 현재 날짜를 빼면 됩니다.

❻ 정상 대여 기간을 초과한 상태로, 연체료를 누적해 출력합니다. 연체가 발생하면 누적 연체료 변수 fine에 하루당 500원을 더합니다. 이 값은 반복문이 실행될 때마다 계속 누적됩니다.

셀프체크

chap_06/_SelfCheck_06.java

```
public class _SelfCheck_06 {
    public static void main(String[] args) {
        int currentLevel = 72; // 1. 현재 잔량 설정
        System.out.println("충전을 시작합니다."); // 2. 충전 시작 안내문 출력
        // 3. 현재 잔량부터 100%까지 반복 수행
        for (int i = currentLevel; i <= 100; i++) {
            System.out.println("현재 충전량: " + i + "%");
            // 특정 구간(20, 50, 80) 확인
            if (i == 20 || i == 50 || i == 80) {
                System.out.println("충전 중입니다.");
            }
        }
        // 4. 100% 도달 시 출력
        System.out.println("충전이 완료되었습니다. 케이블을 분리해 주세요.");
    }
}
```

7장

1분 퀴즈

1. ④ 배열은 같은 자료형의 데이터를 메모리의 연속적인 공간에 저장하는 구조입니다. 이 구조 덕분에 인덱

스를 통해 빠르고 효율적으로 데이터에 접근할 수 있습니다.

2. ① 배열의 인덱스는 0부터 시작합니다. 따라서 첫 번째 요소는 항상 인덱스 0, 즉 `array[0]`으로 접근합니다.

3. ① 기본 `for` 문은 인덱스 변수를 사용해 배열 요소에 순서대로 접근할 수 있습니다. 인덱스 값이 필요한 특정 작업에 유용합니다.

4. ② 2차원 배열에서 첫 번째 인덱스는 행(줄), 두 번째 인덱스는 열(칸)입니다. 따라서 두 번째 행은 인덱스 1, 세 번째 열은 인덱스 2이므로 `matrix[1][2]`로 접근합니다.

5. ③ 2차원 배열은 행과 열로 구성되므로 일반적으로 바깥 반복문은 행을 순회하고, 안쪽 반복문은 각 행의 열을 순회하는 중첩 `for` 문을 사용합니다.

실습 문제

chap_07/_Quiz_07.java

```
public class _Quiz_07 {
    public static void main(String[] args) {
        String[] favoriteFoods = {"떡볶이", "치킨", "파스타"}; ---------- ❶
        System.out.println("--- 친구의 최애 음식 ---"); ---------- ❷
        for (int i = 0; i < favoriteFoods.length; i++) { ---------- ❸
            System.out.println((i + 1) + ". " + favoriteFoods[i]); ------- ❹
        }
    }
}
```

❶ 친구가 좋아하는 음식 3가지를 하나의 배열로 저장합니다. 이렇게 하면 같은 종류의 데이터를 하나의 이름으로 묶어 관리할 수 있어 여러 개의 변수를 따로 만들 필요가 없습니다.

❷ 제목을 한 줄 출력합니다.

❸ `for` 문을 사용해 배열에 들어 있는 음식 이름을 하나씩 출력합니다. `i`는 0부터 시작하고, `favoriteFoods.length`는 배열의 길이(칸 수)인 3을 의미합니다. 따라서 `i`는 0, 1, 2 순으로 순회합니다.

❹ 배열의 인덱스는 0부터 시작하지만, 사람에게 익숙한 시작 번호는 1입니다. 따라서 인덱스를 나타내는 `i`를 사용해 `(i + 1)` 형태로 번호를 붙여 음식 이름과 함께 출력합니다.

셀프체크

chap_07/_SelfCheck_07.java

```
public class _SelfCheck_07 {
```

```
    public static void main(String[] args) {
        // 기내용 수하물 물품 6개 설정
        String[] items = {"옷", "지갑", "노트북", "이어폰", "스프레이", "비타민"};
        boolean found = false; // 발견 여부를 저장할 변수
        for (int i = 0; i < items.length; i++) { // 배열 순회 및 검사
            String item = items[i];
            // 금지 물품(스프레이, 공구) 조건 확인
            if (item.equals("스프레이") || item.equals("공구")) {
                System.out.println(item + "가 발견되었습니다. 확인해 주세요.");
                found = true; // 발견 상태로 변경
                break; // 검사 중단
            }
        }
        // 모든 검사를 마친 후 결과 처리
        if (!found) { // found가 false인 경우(금지 물품이 없음)
            System.out.println("금지 물품이 발견되지 않았습니다. 안전하게 탑승하세요.");
        }
    }
}
```

8장

1분 퀴즈

1. ④ 메서드는 관련된 코드를 하나로 묶어 중복을 줄이고, 프로그램의 구조를 명확하게 만들어 가독성과 유지보수성을 높여줍니다.

2. ③ 메서드는 선언할 때 매개변수를 정의하고, 호출할 때 이 매개변수를 통해 값을 전달받습니다.

3. ② return 키워드는 메서드의 실행을 종료하고, 계산된 결과를 메서드를 호출한 위치로 반환하는 역할을 합니다.

4. ② 메서드 오버로딩은 메서드 이름이 같아도 매개변수의 자료형이나 개수가 다르면 가능합니다. 반환형이나 메서드 이름은 오버로딩 조건이 아닙니다.

5. ④ 메서드 안에 선언한 변수는 지역 변수로, 해당 메서드의 중괄호 블록 안에서만 접근할 수 있습니다.

실습 문제

이 문제의 핵심은 기능별로 메서드를 명확히 나누고, main 메서드에서 이들을 조합해 사용하는 구조를 이해

하는 데 있습니다. 하나의 메서드 안에 모든 코드를 작성하는 것보다 역할을 분리하면 코드를 훨씬 효율적으로 관리할 수 있습니다.

chap_08/_Quiz_08.java

```
public class _Quiz_08 {
    public static void main(String[] args) { ------------------------------------ ❶
        int totalCost = 50000;    // 총 주유 금액
        int pricePerLiter = 1600; // 리터당 휘발유 가격
        double distance = 620;    // 총 주행 거리(km)
        // 메서드 호출
        double fuelAmount = calculateFuelAmount(totalCost, pricePerLiter);
        double fuelEfficiency = calculateFuelEfficiency(distance, fuelAmount);
        // 결과 출력
        System.out.println("--- 연비 계산 결과 ---");
        System.out.println("주유 금액: " + totalCost + "원");
        System.out.println("리터당 가격: " + pricePerLiter + "원");
        System.out.println("총 주유량: " + fuelAmount + "L");
        System.out.println("주행 거리: " + distance + "km");
        System.out.println("--------------------");
        System.out.println("연비: " + fuelEfficiency + " km/L");
    }
    // 총 주유량 계산
    public static double calculateFuelAmount(int totalCost, int pricePerLiter) { ---- ❷
        return (double) totalCost / pricePerLiter;
    }
    // 연비 계산
    public static double calculateFuelEfficiency(double distance, double
fuelAmount) { ------------------------------------------------------------------ ❸
        return distance / fuelAmount;
    }
}
```

❶ main 메서드는 복잡한 계산을 직접 수행하지 않고, 프로그램 전체의 흐름을 관리하는 감독 역할을 맡고 있습니다. 필요한 데이터를 준비하고, 각 메서드에 전달값을 넘기며, 반환된 결과를 받아 최종 결과를 보기 좋은 형태로 출력합니다. 즉, 계산은 각 메서드에게 맡기고, main 메서드는 흐름 제어와 출력에만 집중합니다.

❷ calculateFuelAmount 메서드는 '주유량을 계산한다'는 한 가지 기능만 수행합니다. 총 주유 금액과 리터당 가격이라는 전달값을 받아 계산하고, 총 주유량이라는 결과를 반환합니다. (double) totalCost

/ pricePerLiter 코드에서 (double)로 형 변환을 했기 때문에 50000 / 1600은 정수 31이 아니라 31.25라는 정확한 실수 값으로 계산됩니다.

❸ calculateFuelEfficiency 메서드는 연비를 계산하는 역할을 담당합니다. main 메서드에서 전달받은 주행 거리와, 앞서 calculateFuelAmount 메서드가 계산한 총 주유량을 이용해 최종 연비를 계산하고 그 결과를 반환합니다.

셀프체크

chap_08/_SelfCheck_08.java

```
public class _SelfCheck_08 {
    public static void main(String[] args) {
        int myPerformance = 450000; // 전월 실적 설정
        int discount = getDiscount(myPerformance); // 메서드 호출
        // 결과 출력
        System.out.println("전월 실적: " + myPerformance + "원");
        System.out.println("이번 달 할인 혜택: " + discount + "원");
    }
    public static int getDiscount(int performance) { // 최종 할인 금액 계산
        if (performance >= 600000) {
            return 20000; // 실적 구간 2
        } else if (performance >= 300000) {
            return 10000; // 실적 구간 1
        } else {
            return 0;     // 실적 미달
        }
    }
}
```

9장

1분 퀴즈

1. ④ 클래스는 객체가 가져야 할 속성(변수)과 기능(메서드)을 정의한 설계도입니다. 객체는 이 설계도를 바탕으로 메모리에 생성되는 실체이며, 하나의 클래스로 여러 객체를 만들 수 있습니다.

2. ① 클래스 변수는 static 키워드로 선언하며, 클래스에 소속되어 모든 객체가 하나의 값을 공유합니다. 반면 인스턴스 변수는 객체마다 독립적으로 존재하며 각 객체의 상태를 나타냅니다.

3. ③ 생성자는 new 키워드로 객체를 생성할 때 자동으로 호출되는 특별한 메서드로, 인스턴스 변수 초기화나 객체를 생성할 때 필요한 설정을 담당합니다.

4. ② 캡슐화는 관련 데이터와 기능을 하나의 클래스로 묶어 객체 단위로 관리하는 객체 지향의 기본 원칙입니다. 이때 내부 구현을 외부로부터 숨기는 것을 정보 은닉이라고 합니다. 보통 인스턴스 변수를 private으로 선언해 외부의 직접 접근을 차단하고, 게터와 세터를 통해서만 접근하도록 제한함으로써 이를 구현합니다.

5. ③ public 접근 제어자는 모든 클래스에서 자유롭게 접근할 수 있어 접근 범위가 가장 넓습니다. private은 클래스 내부, default는 같은 패키지, protected는 같은 패키지 또는 다른 패키지의 자식 클래스에서만 접근할 수 있습니다.

실습 문제

chap_09/_Quiz_09.java

```
public class _Quiz_09 {
    public static void main(String[] args) {
        // ClassSlot 클래스로부터 객체 생성 ------------------------------------ ❹
        ClassSlot slot1 = new ClassSlot("수요일", "09:00", "10:00");
        ClassSlot slot2 = new ClassSlot("목요일", "14:00", "15:00");
        ClassSlot slot3 = new ClassSlot("금요일", "17:00", "18:00");
        System.out.println("--- 보충 수업 가능 시간 ---");
        // 각 객체의 정보 출력 ------------------------------------------------ ❺
        slot1.displayScheduleInfo(1);
        slot2.displayScheduleInfo(2);
        slot3.displayScheduleInfo(3);
    }
}
class ClassSlot {
    // 인스턴스 변수: 데이터 묶음 ------------------------------------------------ ❶
    private String day;
    private String startTime;
    private String endTime;
    public ClassSlot(String day, String startTime, String endTime) { ----- ❷
        this.day = day;
        this.startTime = startTime;
        this.endTime = endTime;
    }
    public void displayScheduleInfo(int index) { ---------------------------- ❸
        System.out.println(index + ". " + this.day + " " + this.startTime
```

```
                + "~" + this.endTime);
    }
}
```

❶ day, startTime, endTime은 각각 요일, 시작 시간, 종료 시간을 의미하는 인스턴스 변수입니다. 모든 변수에 private 접근 제어자를 사용해 외부에서 직접 값을 변경하지 못하도록 보호합니다.

❷ 인스턴스 변수들이 private 접근 제어자를 가지므로 객체를 생성할 때 값을 설정할 수 있도록 생성자를 정의합니다. new ClassSlot(...)이 실행되는 순간 이 생성자가 호출되며, 전달값이 각 인스턴스 변수에 저장됩니다. 이 덕분에 객체는 만들어지는 동시에 데이터를 가지게 됩니다.

❸ displayScheduleInfo 메서드는 객체가 가진 보충 수업 시간 정보를 출력하는 기능을 담당합니다. 출력할 때 필요한 인덱스(index)는 외부로부터 전달받고, 실제 시간 정보는 객체 내부에 저장된 데이터를 사용합니다.

❹ main 메서드에서는 ClassSlot 클래스를 바탕으로 slot1, slot2, slot3라는 서로 독립적인 객체를 3개 생성합니다. 각 객체는 생성자를 통해 자신만의 요일과 시간 정보를 가지게 됩니다.

❺ slot1.displayScheduleInfo(1);처럼 객체를 통해 displayScheduleInfo 메서드를 호출합니다. 이는 "slot1에 1번 시간표를 출력해달라"고 요청하는 것과 같습니다. 각 객체는 자신이 가진 데이터를 바탕으로 결과를 출력합니다.

셀프체크

chap_09/_SelfCheck_09.java

```
class Student {
    String name;
    int score;
    public Student(String name, int score) {
        this.name = name;
        this.score = score;
    }
    public boolean isPassed() {
        if (score >= 60) {
            return true;
        } else {
            return false;
        }
    }
}
```

해설 노트

10장

1분 퀴즈

1. ③ 상속은 '자식 클래스는 부모 클래스다(IS-A)'라는 관계를 의미합니다. 부모 클래스의 변수와 메서드를 물려받아 코드 재사용성을 높이고, 필요에 따라 기능을 확장하거나 재정의할 수 있습니다.
2. ④ 메서드 오버라이딩은 부모 클래스의 메서드를 자식 클래스에서 자신의 필요에 맞게 재정의하는 기법입니다. `@Override` 애너테이션을 사용하면 재정의 여부를 명확히 표현할 수 있습니다.
3. ② 다형성은 부모 클래스 타입의 참조 변수가 여러 자식 클래스 객체를 참조할 수 있게 하여, 하나의 타입으로 다양한 객체를 다룰 수 있게 해주는 개념입니다. 이를 통해 코드의 유연성과 확장성이 크게 향상됩니다.
4. ② `final`은 변경을 제한하는 키워드입니다. 변수에 사용하면 값 변경이 불가능하고, 메서드에 사용하면 오버라이딩을 막으며, 클래스에 사용하면 상속을 금지합니다.
5. ④ 참조 자료형은 실제 데이터가 아닌 객체의 메모리 주소를 저장합니다. 인스턴스 변수로 선언할 때 초기화하지 않으면 기본값은 `null`이며, 변수끼리 값을 대입하면 실제 데이터가 아닌 주소값이 복사됩니다.

실습 문제

chap_10/_Quiz_10.java

```
public class _Quiz_10 {
    public static void main(String[] args) {
        // 다형성을 활용한 배열 생성
        Animal[] animals = new Animal[2]; ------------------------------- ❻
        animals[0] = new Lion();
        animals[1] = new Penguin();
        System.out.println("동물들의 정보를 출력합니다.");
        System.out.println("-----------------");
        for (Animal animal : animals) { --------------------------------- ❼
            animal.printInfo();
            System.out.println("-----------------");
        }
        System.out.println("정보 출력이 완료되었습니다.");
    }
}
// 부모 클래스: 모든 동물의 공통 속성(name)과 기능(printInfo)을 정의
class Animal { ------------------------------------------------------------ ❶
```

```
    public String name;
    // 생성자: 이름을 받아 초기화
    public Animal(String name) {
        this.name = name;
    }
    // 공통 기능: 이름을 출력
    public void printInfo() {
        System.out.println("이름: " + this.name);
    }
}
// 자식 클래스 1: Animal을 상속
class Lion extends Animal { ---------------------------------------------- ❷
    // 생성자: 부모 생성자를 호출해 이름을 "사자"로 설정
    public Lion() {
        super("사자");
    }
    // 메서드 오버라이딩(재정의)
    @Override
    public void printInfo() { ------------------------------------------- ❹
        // 부모의 printInfo()를 먼저 호출(이름 출력)
        super.printInfo();
        // Lion만의 기능 추가
        System.out.println("특징: 아프리카의 초원에서 살며, 멋진 갈기를 가지고 있습니다.");
        System.out.println("울음소리: 낮고 힘 있는 소리를 냅니다.");
    }
}
// 자식 클래스 2: Animal을 상속
class Penguin extends Animal { ------------------------------------------- ❸
    // 생성자: 부모 생성자를 호출해 이름을 "펭귄"으로 설정
    public Penguin() {
        super("펭귄");
    }
   // 메서드 오버라이딩(재정의)
    @Override
    public void printInfo() { ------------------------------------------- ❺
        // 부모의 printInfo()를 먼저 호출 (이름 출력)
        super.printInfo();
        // Penguin만의 기능 추가
        System.out.println("특징: 남극에 살며, 날지 못하지만 수영을 아주 잘합니다.");
        System.out.println("울음소리: 짧고 날카로운 소리를 냅니다.");
```

```
    }
}
```

❶ Animal 클래스는 모든 동물이 공통으로 가지는 이름(name)과 이름을 출력하는 기능(printInfo)을 정의합니다.

❷❸ Lion과 Penguin 클래스는 extends 키워드로 Animal을 상속받아 name과 printInfo()를 그대로 물려받습니다. 또한 두 클래스는 각각 생성자에서 super("사자"), super("펭귄")을 호출해 부모 생성자에게 초기화(이름 설정)를 맡깁니다.

❹❺ printInfo 메서드는 자식 클래스에서 오버라이딩합니다. 이때 핵심은 super.printInfo();입니다. 부모의 출력(이름 출력) 로직은 재사용하고, 그 아래에 각 동물만의 특징과 울음소리를 추가합니다.

❻ Animal[] animals 배열은 부모 타입(Animal) 하나로 서로 다른 자식 객체(Lion, Penguin)를 담습니다. 부모 타입의 변수가 자식 타입의 객체를 가리킬 수 있는 것이 바로 다형성입니다.

❼ 다형성 덕분에 for (Animal animal : animals) 반복문이 가능해집니다. 반복문을 돌며 animal.printInfo()를 호출하면, 자바는 실제 객체가 Lion인지 Penguin인지에 따라 각 클래스에서 재정의한 printInfo 메서드를 찾아 실행합니다.

셀프체크

chap_10/_SelfCheck_10.java

```
public class _SelfCheck_10 { (중략) }

class Payment { (중략) }

class CardPayment extends Payment { // 카드 결제
    @Override
    public void pay(int amount) {
        System.out.println("카드로 " + amount + "원을 결제했습니다.");
    }
}
class CashPayment extends Payment { // 현금 결제
    @Override
    public void pay(int amount) {
        System.out.println("현금으로 " + amount + "원을 결제했습니다.");
    }
}
class NadoPayPayment extends Payment { // 나도페이 결제
```

```
    @Override
    public void pay(int amount) {
        System.out.println("나도페이로 " + amount + "원을 결제했습니다.");
    }
}
```

11장

1분 퀴즈

1. ③ 추상 클래스는 미완성된 설계도이기 때문에 직접 객체를 생성할 수 없습니다. 추상 클래스를 상속받아 모든 추상 메서드를 구현한 자식 클래스만 객체로 생성할 수 있습니다.
2. ① 추상 메서드는 선언만 있고 몸통이 없는 메서드입니다. 이를 상속받은 클래스는 반드시 해당 메서드를 구현해야 합니다. 그래야 객체를 생성할 수 있는 완성된 클래스가 됩니다.
3. ③ 인터페이스는 클래스가 여러 기능을 구현할 수 있도록 설계된 문법 요소입니다. 단일 상속의 한계를 보완해 다양한 역할을 동시에 수행할 수 있게 해줍니다.
4. ② 인터페이스에 선언한 메서드는 기본으로 public abstract로 간주합니다. 명시적으로 작성하지 않아도 컴파일러가 자동으로 적용하며, 이를 구현하는 클래스는 반드시 해당 메서드를 구현해야 합니다.

실습 문제

chap_11/_Quiz_11.java

```
interface SkillUser { // 스킬 사용이 가능한 능력을 정의하는 인터페이스
    void useSkill(GameCharacter target);
}
```

SkillUser는 스킬을 사용할 수 있는 능력을 의미하는 인터페이스로, useSkill() 하나만을 선언하고 있습니다. 이 인터페이스를 구현한 클래스는 '스킬 사용이 가능하다'라는 기능적 약속을 갖게 됩니다.

chap_11/_Quiz_11.java

```
abstract class GameCharacter { // 모든 게임 캐릭터의 부모가 될 추상 클래스
    protected String name;
    protected int hp;
    protected int power;
    // 생성자: 이름, 체력, 공격력을 초기화
```

```
    public GameCharacter(String name, int hp, int power) {
        this.name = name;
        this.hp = hp;
        this.power = power;
    }
    public abstract void attack(GameCharacter target); // 공격하는 기능
    public void takeDamage(int damage) { // 피해를 입는 기능
        this.hp -= damage;
        System.out.println(">> " + this.name + "가 " + damage
            + "의 피해를 입었습니다.(남은 체력: " + this.hp + ")");
    }
    public void showStatus() { // 현재 상태를 보여주는 기능
        System.out.println("상태) " + this.name + ", 남은 체력: " + this.hp);
    }
}
```

GameCharacter는 모든 캐릭터가 공통적으로 가지는 요소를 묶어둔 기본 설계도이자 청사진 역할을 합니다. 문제의 조건에 맞게 name, hp, power라는 인스턴스 변수를 포함하며, 피해를 처리하는 takeDamage()와 현재 상태를 출력하는 showStatus() 메서드는 이미 구현 상태입니다.

반면, attack()은 추상 메서드로 선언되었습니다. 이는 "모든 캐릭터는 공격을 할 수 있어야 한다"라는 규칙은 강제하되, "어떻게 공격할 것인지는 각 캐릭터에 맡기겠다"라는 의미입니다. 기사는 검으로 공격하고, 마법사는 마법으로 공격하며, 이후 등장할 다른 캐릭터 역시 각자의 방식으로 공격할 수 있도록 확장 가능성을 열어둔 설계입니다.

chap_11/_Quiz_11.java

```
class Knight extends GameCharacter { // GameCharacter를 상속받는 Knight 클래스
    // 생성자: 부모 클래스의 생성자를 호출해 초기화
    public Knight(String name, int hp, int power) {
        super(name, hp, power);
    }
    @Override
    public void attack(GameCharacter target) { // 추상 메서드 구현: 기본 검 공격
        System.out.println(this.name + "가 " + target.name + "를 검으로 공격합니다!");
        target.takeDamage(this.power);
    }
}
```

기사 역할을 하는 Knight 클래스는 GameCharacter만 상속합니다. 부모 클래스로부터 기본 캐릭터의 공통 기능을 물려받고, 추상 메서드였던 attack()을 검 공격 방식으로 구현했습니다. 공격할 때 상대방의 체력이 감소하도록 takeDamage()를 호출하는 부분까지 포함되어 하나의 완성된 전투 행동을 수행합니다.

chap_11/_Quiz_11.java

```
// GameCharacter를 상속하고 SkillUser 인터페이스를 구현하는 Wizard 클래스
class Wizard extends GameCharacter implements SkillUser {
    // 생성자: 부모 클래스의 생성자를 호출해 초기화
    public Wizard(String name, int hp, int power) {
        super(name, hp, power);
    }
    @Override
    public void attack(GameCharacter target) { // 추상 메서드 구현: 기본 마법 공격
        System.out.println(this.name + "가 " + target.name
                + "를 기본 마법으로 공격합니다!");
        target.takeDamage(this.power);
    }
    @Override
    public void useSkill(GameCharacter target) { // 인터페이스 메서드 구현: 스킬 공격
        System.out.println(this.name + "가 " + target.name
                + "에게 윈드 블레이드 스킬을 사용합니다!");
        target.takeDamage(this.power * 2);
    }
}
```

마법사 역할을 하는 Wizard 클래스는 구조가 조금 더 복잡합니다. Wizard는 GameCharacter를 상속함과 동시에 SkillUser 인터페이스를 구현합니다. 그 결과 Wizard 클래스에는 두 가지 책임이 생깁니다.

- GameCharacter로부터 물려받은 추상 메서드 attack() 구현
- SkillUser 인터페이스가 요구하는 useSkill() 구현

attack()은 기본 마법 공격으로 구현되었고, useSkill()은 공격력의 2배만큼 피해를 주는 강력한 스킬로 구현되었습니다. 문제 조건에 맞게 power * 2를 적용한 부분입니다.

시뮬레이션 코드에서 wizard.useSkill(knight)와 같은 호출이 가능했던 이유도 바로 여기에 있습니다. Wizard는 SkillUser 인터페이스를 구현했기 때문에 useSkill() 메서드를 사용할 수 있지만, Knight는 이 인터페이스를 구현하지 않았기 때문에 스킬 사용 기능 자체가 존재하지 않습니다.

이 예제의 핵심은 설계 방식에 있습니다. 공통 구조와 기본 동작은 추상 클래스로 묶고, 특정 상황에서만 필요한 선택 기능은 인터페이스로 분리해 부착하는 방식입니다. 이렇게 설계해두면, 이후에 궁수, 도적, 힐러 같

은 새로운 캐릭터를 추가하더라도 필요한 인터페이스만 조합해 구현하면 되므로 코드를 거의 수정하지 않고도 유연하게 기능을 확장할 수 있습니다.

셀프체크

chap_11/_SelfCheck_11.java

```
public class _SelfCheck_11 {
    public static void main(String[] args) { (중략) }
}
abstract class BankAccount { (중략) }
class SavingsAccount extends BankAccount { // 적금 통장 클래스
    @Override
    void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(amount + "원 출금 완료(현재 잔액: " + balance + "원)");
        } else {
            System.out.println("잔액이 부족합니다.(현재 잔액: " + balance + "원)");
        }
    }
}
class MinusAccount extends BankAccount { // 마이너스 통장 클래스
    @Override
    void withdraw(int amount) {
        balance -= amount;
        System.out.println(amount + "원 출금 완료(현재 잔액: " + balance + "원)");
    }
}
```

12장

1분 퀴즈

1. ③ 컬렉션 프레임워크는 배열과 달리 크기가 동적으로 변할 수 있으며, 추가, 삭제, 검색, 정렬 등 데이터를 다루기 위한 다양한 편의 메서드를 제공합니다.
2. ④ List 계열 컬렉션은 데이터를 추가한 순서를 인덱스로 관리하며, 동일한 값을 여러 번 저장하는 중복을 허용합니다.

3. ④ Set은 중복을 허용하지 않는 컬렉션이며, 그중 HashSet은 데이터의 저장 순서를 보장하지 않습니다.

4. ③ HashMap은 키와 값을 한 쌍으로 저장하는 자료구조로, 키는 중복될 수 없습니다. 동일한 키로 값을 다시 저장하면 기존 값은 새로운 값으로 덮어씌워집니다.

5. ④ LinkedList는 요소들이 서로 연결된 구조이므로 중간이나 양 끝에서의 삽입, 삭제 연산이 ArrayList보다 효율적일 수 있습니다.

6. ② hasNext()는 Iterator에서 다음 요소가 존재하는지 확인할 때 사용합니다. 다음 요소가 있으면 true, 더 이상 요소가 없으면 false를 반환합니다. 일반적으로 while(iterator.hasNext()) 형태를 사용해 컬렉션의 모든 요소를 순차적으로 순회합니다.

실습 문제

chap_12/_Quiz_12.java

```
import java.util.ArrayList;

class WishlistItem { ------------------------------------------------------ ❶
    public String name;
    public int price;
    public WishlistItem(String name, int price) {
        this.name = name;
        this.price = price;
    }
}
public class _Quiz_12 {
    public static void main(String[] args) {
        ArrayList<WishlistItem> wishlist = new ArrayList<>(); --------- ❷
        wishlist.add(new WishlistItem("노트북", 150)); ------------------ ❸
        wishlist.add(new WishlistItem("모니터", 30));
        wishlist.add(new WishlistItem("기계식 키보드", 24));
        wishlist.add(new WishlistItem("무선 마우스", 7));
        wishlist.add(new WishlistItem("노트북 파우치", 4));
        wishlist.add(new WishlistItem("USB 허브", 3));
        wishlist.add(new WishlistItem("마우스 패드", 1));
        System.out.println("--- 10만 원 초과 상품 목록 ---");
        for (WishlistItem item : wishlist) { ---------------------------- ❹
            if (item.price > 10) {
                System.out.println(item.name + "(" + item.price + "만 원)");
            }
        }
```

```
        int totalPrice = 0; ---------------------------------------------- ❺
        for (WishlistItem item : wishlist) { ---------------------------- ❻
            totalPrice += item.price; // 각 아이템의 price 변수에 접근해 합산
        }
        System.out.println("--- 총 합계 ---");
        System.out.println(totalPrice + "만 원"); ----------------------- ❼
    }
}
```

❶ 위시리스트에 담길 상품의 이름과 가격 정보를 표현하기 위해 WishlistItem 클래스를 정의합니다. 이 클래스는 상품명(name)과 가격(price)을 인스턴스 변수로 가지며, 생성자를 통해 두 값을 전달받아 초기화하는 간단한 데이터 클래스입니다.

❷ 위시리스트를 관리하기 위해 ArrayList<WishlistItem> 타입의 리스트를 생성합니다. 제네릭을 사용하므로 이 리스트에는 WishlistItem 객체만 저장할 수 있습니다.

❸ 예시로 주어진 상품들을 new 키워드를 사용해 객체로 생성하고, add()를 사용해 리스트에 차례대로 추가합니다. 이때 price 값은 만 원 단위로 저장합니다. 가격을 계산하고 출력할 때 단위를 통일하면 연산이 훨씬 단순해지기 때문입니다.

❹ for-each 반복문으로 위시리스트를 순회하면서 가격이 10만 원을 초과하는 상품만 출력합니다. 각 객체가 가진 price 값을 기준으로 if 조건문을 적용해 필요한 상품만 골라냅니다.

❺ 전체 상품 가격의 총합을 계산하기 위해 정수형 변수 totalPrice를 0으로 초기화합니다.

❻ 다시 한번 for-each 반복문으로 각 상품의 price 값을 누적해 더합니다. 이렇게 하면 위시리스트에 담긴 모든 상품의 총금액을 손쉽게 계산할 수 있습니다.

❼ 마지막으로 계산된 총합을 출력합니다.

셀프체크

chap_12/_SelfCheck_12.java

```
import java.util.HashSet;

public class _SelfCheck_12 {
    public static void main(String[] args) {
        // 1. HashSet 생성
        HashSet<String> participantSet = new HashSet<>();
        // 2. 응모자 데이터 추가
```

```
        participantSet.add("김가영");
        participantSet.add("이승철");
        participantSet.add("정다혜");
        participantSet.add("김가영"); // 중복 데이터
        participantSet.add("송재승");
        // 3. 최종 인원 확인
        System.out.println("최종 응모 인원: " + participantSet.size() + "명");
        // 4. 특정 인물 확인
        if (participantSet.contains("이승철")) {
            System.out.println("이승철 님 응모 확인 완료");
        }
        // 5. 명단 전체 출력
        System.out.println("--- 당첨 후보자 명단(순서 무관) ---");
        for (String name : participantSet) {
            System.out.println(name);
        }
    }
}
```

13장

1분 퀴즈

1. ④ try-catch 문은 프로그램 실행 중 발생하는 예외를 잡아내어 프로그램이 비정상적으로 종료되는 것을 방지합니다. 예외 상황을 처리한 뒤에도 프로그램이 계속 실행될 수 있도록 만드는 것이 주된 목적입니다.

2. ③ finally 블록은 예외 발생 여부와 관계없이 try 또는 catch 블록을 실행한 후 반드시 실행됩니다. 파일 닫기, 자원 반납과 같은 마무리 작업에 주로 사용합니다.

3. ③ throw 키워드는 실행 중 새로운 예외 객체를 생성하고, 해당 예외를 즉시 발생시킬 때 사용합니다.

4. ② try-with-resources는 AutoCloseable 인터페이스를 구현한 객체를 try 괄호 안에 선언하면, try 블록이 종료되는 시점에 예외 발생 여부와 관계없이 자동으로 close()를 호출해 자원을 반납합니다. 이로 인해 finally 블록 없이도 안전하고 간결한 자원 관리가 가능합니다.

5. ③ 사용자 정의 예외는 기존 예외만으로는 표현하기 어려운 규칙 위반 등을 명확하게 나타내기 위해 사용합니다. 이를 사용하면 코드의 가독성과 유지보수성이 크게 향상됩니다.

실습 문제

chap_13/_Quiz_13.java

```
class InsufficientFundsException extends Exception { -------- ❶
    public InsufficientFundsException(String message) {
        super(message);
    }
}
class VendingMachine { ---------------------------------------- ❷
    public void purchase(String item, int price, int moneyInserted)
            throws InsufficientFundsException {
        System.out.println("--- 구매 시도 ---");
        System.out.println("상품: " + item + " / 가격: " + price
                + "원 / 투입 금액: " +  moneyInserted + "원");
        if (moneyInserted < price) { // 투입 금액이 가격보다 적으면 예외 발생
            throw new InsufficientFundsException("금액이 부족합니다.");
        }
        // 정상 구매 처리
        int change = moneyInserted - price;
        System.out.println("[성공] " + item + " 구매 완료! 거스름돈은 "
                + change + "원입니다.");
    }
}
public class _Quiz_13 { ---------------------------------------- ❸
    public static void main(String[] args) {
        VendingMachine machine = new VendingMachine();
        try { // 정상적인 경우(금액 충분)
            machine.purchase("콜라", 1200, 2000);
        } catch (InsufficientFundsException e) {
            System.out.println("[오류] " + e.getMessage());
        }
        System.out.println();
        try { // 예외가 발생하는 경우(금액 부족)
            machine.purchase("사이다", 1500, 1000);
        } catch (InsufficientFundsException e) {
            System.out.println("[오류] " + e.getMessage());
        }
    }
}
```

❶ 이 클래스는 금액이 부족한 상황을 명확하게 표현하기 위해 만든 사용자 정의 예외입니다. Exception 클래스를 상속받았기 때문에 예외가 발생할 가능성이 있는 코드는 반드시 try-catch 문 또는 throws로 처리해야 합니다. 이는 자바가 예외 처리 책임을 컴파일 단계에서 강제하도록 만드는 중요한 약속이며, 프로그램의 안정성을 크게 높여줍니다.

❷ VendingMachine 클래스는 실제 음료 구매 과정을 담당합니다. purchase 메서드는 상품명, 가격, 투입 금액을 전달받아 구매를 시도합니다. 투입 금액이 가격보다 적으면 정의한 규칙을 위반한 상황이므로 throw 키워드로 InsufficientFundsException을 직접 발생시킵니다. 조건을 통과하면 거스름돈을 계산하고 구매 성공 메시지를 출력합니다. 예외 흐름(금액 부족)과 정상 흐름(구매 성공)이 명확히 분리되어 있습니다.

❸ main 메서드는 자판기를 실제로 사용하는 역할을 합니다. VendingMachine 객체를 생성한 뒤, 구매 성공과 구매 실패 두 가지 시나리오를 각각 실행합니다. 예외가 발생할 수 있는 purchase() 호출 부분을 try-catch 문으로 감싸 프로그램이 중단되지 않도록 안전하게 처리합니다. 특히 catch (InsufficientFundsException e) 블록에서는 예외 객체에 담긴 메시지(e.getMessage())를 활용해 사용자에게 구체적인 오류 안내를 제공합니다.

셀프체크

chap_13/_SelfCheck_13.java

```
public class _SelfCheck_13 {
    public static void main(String[] args) {
        int totalAmount = 50000; // 총 결제 금액
        int people = 4; // 총 인원 수
        try {
            int amountPerPerson = totalAmount / people; // 여기서 0이면 예외 발생
            System.out.println("총 결제 금액 " + totalAmount
                    + "원에서 1인당 지불 금액은 " + amountPerPerson + "원입니다.");
        } catch (ArithmeticException e) {
            System.out.println("오류: 0명으로는 나눌 수 없습니다.");
        } finally {
            System.out.println("정산 프로그램을 종료합니다.");
        }
    }
}
```

14장

1분 퀴즈

1. ② next()는 공백(띄어쓰기, 탭, 줄 바꿈 등)을 구분자로 사용해 공백 전까지 한 단어만 읽습니다. 반면 nextLine()은 줄 바꿈([Enter]) 전까지 한 줄 전체를 읽습니다.
2. ③ %.2f는 실수를 소수점 이하 둘째 자리까지 표시하라는 의미입니다. 이때 셋째 자리에서 반올림이 이루어집니다. 3.14159를 소수점 둘째 자리까지 반올림하면 3.14가 됩니다. 또한 %n은 운영체제와 관계없이 줄 바꿈을 수행하는 서식 지정자입니다.
3. ④ mkdir()은 단일 폴더만 생성하지만, mkdirs()는 필요한 경우 상위 폴더까지 모두 생성합니다. 여러 단계의 폴더 구조를 만들 때 유용합니다.
4. ① BufferedReader는 내부 버퍼를 사용해 파일을 읽는 클래스로, readLine 메서드를 사용해 파일 내용을 한 줄씩 읽을 수 있습니다.
5. ② isFile()은 해당 경로가 일반 파일인지 여부를 boolean 값(true/false)으로 반환하며, 파일과 폴더를 구분할 때 사용합니다. 참고로, 폴더인지 확인하려면 isDirectory()를 사용합니다.

실습 문제

chap_14/_Quiz_14.java

```
import java.io.BufferedReader; // 줄 단위로 텍스트를 읽기 위한 클래스
import java.io.FileReader;     // 파일에서 문자 스트림을 읽기 위한 클래스
import java.io.IOException;    // 입출력 예외 처리를 위한 클래스

public class _Quiz_14 {
    public static void main(String[] args) {
        String fileName = "vacation.txt"; ---------------------------- ❶
        System.out.println("수상한 여행 편지에서 암호 해독을 시작합니다.");
        System.out.println("------------------------------------");
        try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
---------------------------------------------------------------------- ❷
            String line; // 읽어온 한 줄을 저장할 변수
            while ((line = br.readLine()) != null) { ---------------- ❸
                String firstLetter = line.substring(0, 1); -------- ❹
                System.out.print(firstLetter); ---------------------- ❺
            }
            System.out.println();
```

```
            System.out.println("------------------------------------");
            System.out.println("암호 해독 완료! 축하합니다!");
        } catch (IOException e) { // 파일이 없거나 읽기 권한이 없을 때 예외 처리
            System.err.println("파일을 읽는 중 오류가 발생했습니다: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

❶ vacation.txt 파일명을 변수에 저장합니다.

❷ 파일을 효율적으로 읽기 위해 BufferedReader와 FileReader를 사용합니다. try-with-resources 구문 안에 객체를 선언해 try 블록이 종료되면 자바가 자동으로 close()를 호출합니다.

❸ while 문의 조건은 "파일에서 한 줄을 읽어 line 변수에 저장하고, 그 값이 null이 아닐 동안 반복하라"는 의미입니다. readLine()은 파일의 끝에 도달하면 null을 반환합니다.

❹ 각 줄에서 substring(0, 1)을 사용해 첫 번째 글자를 추출합니다. 이는 문자열의 0번 인덱스부터 1번 인덱스 직전까지 잘라내므로 정확히 한 글자를 반환합니다.

❺ System.out.print()를 사용해 줄 바꿈 없이 글자를 이어서 출력합니다. 그 결과 각 줄의 첫 글자가 한 줄로 모여 숨겨진 메시지가 완성됩니다.

셀프체크

chap_14/_SelfCheck_14.java

```
import java.util.Scanner;
import java.time.LocalDate; // 날짜를 다루는 클래스
import java.time.format.DateTimeFormatter; // 날짜 형식을 지정하는 클래스

public class _SelfCheck_14 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // 이름 입력받기
        System.out.print("수료자 이름을 입력하세요: ");
        String name = sc.nextLine();
        // 오늘 날짜 가져오기 및 형식 지정
        LocalDate today = LocalDate.now(); // 오늘 날짜 정보
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy년 M월 d일");
        String formattedDate = today.format(formatter); // 지정한 형식으로 변환
        // 수료증 출력
```

```
        System.out.println();
        System.out.println("========================================");
        System.out.println("              [ 수 료 증 ]              ");
        System.out.println("========================================");
        System.out.println("  성명: " + name);
        System.out.println("  과정명: 나도코딩 자바 기본편");
        System.out.println();
        System.out.println("  내용: 위 사람은 자바의 핵심 기초 과정을");
        System.out.println("        성실히 이수하였기에 이 수료증을");
        System.out.println("        수여합니다.");
        System.out.println();
        System.out.println("            " + formattedDate); // 오늘 날짜
        System.out.println("            나도코딩 드림              ");
        System.out.println("========================================");
    }
}
```

부록 A

1분 퀴즈

1. ④ 제네릭은 클래스나 메서드에서 사용할 타입을 미리 지정해 다양한 타입을 하나의 코드로 처리하면서 타입 안전성을 높이고 불필요한 형 변환을 줄이기 위해 사용합니다.
2. ③ 제네릭 클래스를 사용하면 변수를 선언할 때 <타입>을 지정해야 하고, 객체를 생성할 때는 다이아몬드 연산자(<>)를 사용해 타입을 자동으로 추론할 수 있습니다.
3. ② 제네릭은 객체 타입(참조 자료형)만 사용할 수 있기 때문에 int, double과 같은 기본 자료형은 직접 사용할 수 없습니다. 따라서 제네릭에서는 int 대신 Integer, double 대신 Double과 같은 래퍼 클래스를 사용해야 합니다.

부록 B

1분 퀴즈

1. ④ 익명 클래스는 이름이 없는 클래스로, 특정 객체를 생성하는 시점에 한 번만 사용하기 위해 정의합니다. 주로 메서드를 일시적으로 오버라이드하거나 인터페이스를 구현할 때 사용합니다.

2. ② 함수형 인터페이스는 람다식의 타입이 되는 인터페이스로, 구현해야 할 추상 메서드가 오직 하나인 인터페이스를 말합니다. `@FunctionalInterface` 애너테이션을 붙이면 컴파일러가 조건을 자동으로 검사해 주지만, 이는 필수가 아닌 선택 사항입니다. 추상 메서드가 하나이기만 하면 애너테이션 없이도 함수형 인터페이스로 사용할 수 있습니다.

3. ① 람다식은 매개변수 목록, 화살표(->), 실행할 코드(본문)로 구성됩니다.

4. ④ 중간 연산(`filter()`, `map()`, `sorted()` 등)은 새로운 스트림을 반환하고, 최종 연산(`forEach()`, `count()`, `collect()` 등)이 호출되기 전까지 실제로 실행되지 않습니다. 최종 연산이 실행되는 순간 스트림의 데이터를 사용해 결과를 반환합니다. 이때 사용한 스트림은 재사용할 수 없습니다.

실습 문제

appendix_02/_Quiz_a2.java

```
import java.util.ArrayList;

class Task { ------------------------------------------------------------------ ❶
    public String title;
    public String category;
    public boolean isCompleted;
    public Task(String title, String category, boolean isCompleted) {
        this.title = title;
        this.category = category;
        this.isCompleted = isCompleted;
    }
}
public class _Quiz_a2 {
    public static void main(String[] args) {
        ArrayList<Task> todoList = new ArrayList<>(); ------------------------- ❷
        todoList.add(new Task("자바 강의 듣기", "공부", true)); ----------------- ❸
        todoList.add(new Task("이메일 회신하기", "업무", false));
        todoList.add(new Task("운동하기", "개인", false));
        todoList.add(new Task("주간 보고서 작성", "업무", false));
        todoList.add(new Task("알고리즘 문제 풀기", "공부", false));
        todoList.add(new Task("팀 미팅 준비", "업무", true));
        System.out.println("--- 미완료된 업무 목록 ---");
        todoList.stream() ------------------------------------------------------ ❹
                .filter(task -> task.category.equals("업무")) ---------------- ❺
                .filter(task -> !task.isCompleted) ---------------------------- ❻
                .forEach(task -> System.out.println("- " + task.title)); ---- ❼
```

```
    }
}
```

❶ 할 일 하나를 표현하는 Task 클래스를 정의합니다. title, category, isCompleted 세 개의 인스턴스 변수를 가지며, 객체를 생성할 때 생성자로 모든 값을 초기화합니다.

❷ main 메서드에서 ArrayList<Task>를 생성합니다.

❸ 예시 데이터를 바탕으로 여러 개의 Task 객체를 추가합니다.

❹ 리스트로부터 스트림을 생성합니다.

❺ 카테고리가 "업무"인 Task 객체만 필터링해 다음 단계로 전달합니다.

❻ 전달받은 데이터에서 아직 완료되지 않은 Task 객체만 통과시킵니다.

❼ 마지막으로 남은 할 일 제목을 지정된 형식으로 출력합니다.

셀프체크

appendix_02/_SelfCheck_a2.java

```
interface Button {
    void onClick();
}

public class _SelfCheck_a2 {
    public static void main(String[] args) {
        Button tvButton = new Button() {
            @Override
            public void onClick() {
                System.out.println("TV를 켭니다.");
            }
        };
        Button airconButton = new Button() {
            @Override
            public void onClick() {
                System.out.println("에어컨을 냉방 모드로 가동합니다.");
            }
        };
        tvButton.onClick();
        airconButton.onClick();
    }
}
```

부록 C

1분 퀴즈

1. ④ 프로세스는 실행 중인 프로그램을 의미하며, 스레드는 그 프로세스 안에서 실제 작업 흐름을 담당하는 실행 단위입니다. 하나의 프로세스는 하나 이상의 스레드를 가질 수 있습니다.
2. ③ start()를 호출하면 새로운 스레드를 생성하고 그 스레드에서 run()을 실행합니다. run()을 직접 호출하면 새로운 스레드가 생성되지 않고 현재 스레드에서 일반 메서드처럼 실행됩니다.
3. ② 자바는 클래스의 단일 상속만 허용합니다. Thread 클래스를 상속하면 다른 클래스를 상속할 수 없지만, Runnable 인터페이스를 구현하면 기존 상속 구조를 유지하면서 스레드를 사용할 수 있습니다.
4. ① join()은 특정 스레드가 다른 스레드의 실행이 종료될 때까지 기다리도록 합니다. sleep()은 일정 시간 동안 스레드를 정지시키며, 다른 스레드의 종료 여부와는 관계가 없습니다.
5. ④ synchronized 키워드는 특정 메서드나 코드 블록에 한 번에 하나의 스레드만 접근하도록 제한합니다. 이를 통해 여러 스레드가 동시에 공유 자원을 수정할 때 발생할 수 있는 경쟁 상태를 방지할 수 있습니다.

실습 문제

appendix_03/_Quiz_a3.java

```
class ReadingTask implements Runnable { // 아빠의 할 일을 정의하는 클래스
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("아빠가 " + i + "페이지를 읽고 있습니다.");
            try {
                Thread.sleep(500); // 0.5초 대기
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
        System.out.println("아빠가 책을 다 읽었습니다!");
    }
}
class MathTask implements Runnable { // 딸의 할 일을 정의하는 클래스
    @Override
    public void run() {
```

```
        for (int i = 1; i <= 10; i++) {
            System.out.println("딸이 " + i + "번 문제를 풀고 있습니다.");
            try {
                Thread.sleep(300); // 0.3초 대기
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        }
        System.out.println("딸이 수학 문제를 다 풀었습니다!");
    }
}
public class _Quiz_a3 { // 메인 클래스
    public static void main(String[] args) {
        Thread dadThread = new Thread(new ReadingTask()); -------- ❶
        Thread daughterThread = new Thread(new MathTask()); ------- ❷
        dadThread.start(); ------------------------------------------ ❸
        daughterThread.start(); ------------------------------------- ❹
        try { // 두 사람의 할 일이 끝날 때까지 대기 --------------------- ❺
            dadThread.join();
            daughterThread.join();
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
         System.out.println("아빠와 딸이 보드게임을 시작합니다!"); ----- ❻
    }
}
```

아빠와 딸의 할 일을 정의하는 클래스는 거의 비슷합니다. 모두 Runnable 인터페이스를 구현하고, run() 안에서 반복 횟수, 출력 문구, 대기 시간만 다르게 설정합니다.

❶❷ main 메서드에서는 Thread 객체를 생성해 할 일을 각각 Runnable 객체로 넣어줍니다.

❸❹ start()를 호출해 동시에 할 일을 시작합니다.

❺ join()을 사용해 두 스레드의 작업이 모두 끝날 때까지 메인 스레드가 기다리도록 합니다.

❻ 할 일을 모두 완료하면 보드게임 시작 메시지를 출력합니다.

셀프체크

appendix_03/_SelfCheck_a3.java

```
public class _SelfCheck_a3 {
```

```
    public static void main(String[] args) { (중략) }
}
class Chef extends Thread { // Thread를 상속받는 Chef 클래스
    String dish;
    public Chef(String dish) {
        this.dish = dish;
    }
    @Override
    public void run() { // 스레드가 실행할 작업 정의
        try {
            System.out.println("--- " + dish + " 요리 시작 ---");
            Thread.sleep(1000); // 1초간 요리 중
            System.out.println("--- " + dish + " 요리 완료! ---");
        } catch (InterruptedException e) {
            System.out.println(dish + " 요리 중 문제가 발생했습니다.");
        }
    }
}
```